assignment4_part4

October 24, 2026

1 Assignment 4: Mining Sequence Data (Part IV)

1.1 Application: Sequence Alignment

When looking back at the year 2020, perhaps no one would forget the then rampant coronavirus disease, COVID-19, which had caused 14,366 deaths worldwide as of March 22, 2020. One of the early papers published immediately after the outbreak claims that the coronavirus found in humans is highly (96%) similar in genes to a particular coronavirus found in bats. Based on our previous (perhaps limited) exposure to biology, we know that a gene is a *sequence* of nucleotides that can be expressed in terms of four nucleobases, `A`, `T`, `G` and `C`. To identify similar genes can actually be cast as a *sequence alignment* problem.

In this assignment, we will try to *roughly* verify the claim in the paper by calculating an "alignment score" between the human-hosted coronavirus genes and the bat-hosted coronavirus genes.

1.1.1 Exercise 5. Sequence Alignment (10 pts)

Complete the `seq_align` function to compute the alignment score between the human-hosted coronavirus genes and the bat-hosted coronavirus genes. Specifically, you should

(1) read the two gene sequences from the two files provided and clean them as instructed, and

(2) compute the alignment score **for the first 10,000 nucleobases** using the `pairwise2.align.globalxx` function from the `biopython` package. See its documentation to understand how to calculate such a score.

[18]:
```
from Bio import pairwise2

def seq_align():
    human_seq, bat_seq = None, None

    with open("assets/MN908947.3_human.txt", "r") as human_genes:
        """
        Read the human-hosted coronavirus gene sequence from a file
        and store it in the variable 'human_seq' as a long string.

        * Remeber to strip off the newline character '\n' at the end of each␣
 ↪line.
        * Also, skip the lines with only meta-data, for example:
```

```
        >lcl|MN908947.3_gene_1 [gene=orf1ab] [location=266..21555]␣
↪[gbkey=Gene]
    """

    human_seq = []
    for line in human_genes:
        # YOUR CODE HERE
        #raise NotImplementedError()
        if '[' in line: continue
        human_seq.append(line.replace('\n', ''))
    human_seq = "".join(human_seq)


assert "\n" not in human_seq, '[Exercise 5] Remember to remove the "\n"␣
↪character at the end of each line. '
assert "[" not in human_seq, "[Exercise 5] Remember to remove lines with␣
↪only meta-data. "
assert len(human_seq) == 29132, "[Exercise 5] The length of your human␣
↪genes is not correct. "

with open("assets/MN996532.1_bat.txt", "r") as bat_genes:
    """
    Read the bat-hosted coronavirus gene sequence from a file
    and store it in the variable 'bat_seq' as a long string

    * Remeber to strip off the newline character '\n' at the end of each␣
↪line.
    * Also, skip the lines with only meta-data, for example:
        >lcl|MN908947.3_gene_1 [gene=orf1ab] [location=266..21555]␣
↪[gbkey=Gene]
    """

    bat_seq = []
    for line in bat_genes:
        # YOUR CODE HERE
        #raise NotImplementedError()
        if '[' in line: continue
        bat_seq.append(line.replace('\n', ''))
    bat_seq = "".join(bat_seq)


assert "\n" not in bat_seq, '[Exercise 5] Remember to remove the "\n"␣
↪character at the end of each line. '
assert "[" not in bat_seq, "[Exercise 5] Remember to remove lines with only␣
↪meta-data. "
```

```
    assert len(bat_seq) == 29129, "[Exercise 5] The length of your bat genes is␣
 ↪not correct. "

    # Finally, take the first 10,000 nucleobases (i.e., ATGC) from each␣
 ↪sequence and calculate the alignment score
    # using the pairwise2.align.globalxx function. Use score_only=True.
    score = 0.0
    # YOUR CODE HERE
    #raise NotImplementedError()
    #score = pairwise2.align.globalxx(human_seq[:10000], bat_seq[:10000])[0].
 ↪score
    score = pairwise2.align.globalxx(human_seq[:10000], bat_seq[:10000],␣
 ↪score_only=True)
    return score

#seq_align()
```

[19]:
```
# This code block tests if the `seq_align` function is implemented correctly
# We hide some tests. Passing the displayed assertions does not guarantee full␣
 ↪points.

stu_ans = seq_align()

assert isinstance(stu_ans, float), "[Exercise 5] The alignment score should be␣
 ↪a (float) number. "
assert stu_ans > 9000, "[Exercise 5] The alignment score should be at least␣
 ↪9000. "

del stu_ans
```

[13]:
```
#pairwise2.align.globalxx("ACCGT", "ACG")[0].score
```

Through this vary last assignment in this course, we hope you can see that data-mining techniques, sequence alignment for example, have useful practical applications. We did a rather simple example for pedagogy, but a more elaborate example using BLAST, a professional gene analysis tool, can be found here. Click the "BLAST" button to see the report.

Congratulations that you have finished all assignments of this course! We wish you good luck in the next steps of your journey!